BỘ GIÁO DỤC VÀ ĐÀO TẠO
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI TP. HỒ CHÍ MINH
KHOA KỸ THUẬT ĐIỆN TỬ 2
NGÀNH: CÔNG NGHỆ KỸ THUẬT ĐIỆN, ĐIỆN TỬ



BÁO CÁO ĐỒ ÁN HỌC PHẦN

MÔN: CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

ĐỀ TÀI:

TÌM VỊ TRÍ ĐẦU, CUỐI VÀ TẦN SUẤT CỦA X

Mã đề: DT028

Giảng viên hướng dẫn	:	Hồ Nhựt Minh
Sinh viên thực hiện	:	PHAN ĐỨC HUY
MSSV	:	N24DCDT034
Lớp	:	D24CQDT01-N

TP. HCM, tháng 10 năm 2026

PHIẾU GIAO ĐỀ TÀI

Học phần INT13185 Nhóm 03 Giảng viên Hồ Nhựt Minh

Thông tin	Thông tin đề tài và sinh viên
Họ tên MSSV lớp	Phan Đức Huy — N24DCDT034 — D24CQDT01-N
Mã đề và tên đề	DT028 — Tìm vị trí đầu, cuối và tần suất của x
Căn cứ giáo trình	Chương 2, Mục 2.6.2; mục 2.8 bài 28; trang in 50–52, 62; trang PDF 62–64, 74
Ngày giao ngày nộp	Ngày giao 17/10/2026 — Ngày nộp 20/10/2026

Đề bài: Cho mảng đã tăng có khóa trùng, tìm lần xuất hiện đầu tiên và cuối cùng bằng hai tìm kiếm nhị phân. Từ hai biên đếm số lần xuất hiện, phân biệt với tìm một vị trí bất kỳ và xử lý giá trị hoàn toàn vắng mặt. Trọng tâm là tự viết hàm, theo dõi biến qua từng bước và giải thích kết quả trên các trường hợp nhỏ.

Dữ liệu vào: a::Vector{Int} tăng không giảm, $0 \le n \le 10000$; x nguyên; vắng trả first = last = 0, count = 0. Nhập từ bàn phím hoặc truyền trực tiếp các giá trị, mảng đã khai báo trong chương trình.

Kết quả ra: Hai chỉ số 1-based đầu/cuối và tần suất; nếu thấy, count = last − first + 1; trace hai quá trình tìm biên.

Ví dụ 1: Đoạn chỉ số 2..4 gồm đúng ba số 2.

Ví dụ 2: Không gán vị trí chèn của 3 thành một lần xuất hiện.

Hướng dẫn thực hiện: 1. Đọc mục sách được chỉ dẫn; xác định điều kiện đầu vào, kết quả cần trả và giải ví dụ thứ nhất bằng tay.
2. Lần vết khi gặp a[mid] == x: bản tìm đầu tiếp tục về trái, bản tìm cuối tiếp tục về phải; vẫn lưu vị trí hiện có.
3. Cài hai hàm biên và hàm ghép kết quả; chỉ tính last − first + 1 khi đã xác nhận tìm thấy x.
4. Đối chiếu quét tuyến tính trên mảng nhỏ; thử x ở hai đầu, chiếm cả mảng và không có.
5. Chạy hai ví dụ và năm ca kiểm thử; đối chiếu kết quả mong đợi, sửa lỗi và giải thích một trường hợp biên.

Các hàm nên viết: vi_tri_dau(a,x); vi_tri_cuoi(a,x); tan_suat(a,x).

Năm ca cần kiểm tra: 1. [1, 2, 2, 2, 4], x = 2 cho 2, 4, 3.
2. Cùng mảng, x = 3 cho 0, 0, 0.
3. [5, 5, 5], x = 5 cho 1, 3, 3.

4. [1, 2], x = 1 cho 1, 1, 1.
5. Mảng rỗng cho 0, 0, 0.

Phân tích và nhận xét: Mỗi hàm O(log n), hai hàm vẫn O(log n); vùng phụ O(1). Bản quét tham chiếu O(n). Thử n = 5, 50, 500, 5000 với dãy nhiều trùng; đếm lần xét mid của từng biên riêng.

Yêu cầu thêm sau khi phần cơ bản chạy đúng: Tùy chọn: dùng hai biên để đếm số phần tử nằm trong đoạn giá trị [L, U] hợp lệ.

Sản phẩm: Mã nguồn, các kết quả chạy ví dụ và ca kiểm tra, một lần vết bằng tay, báo cáo theo mẫu môn học. Chạy từ bàn phím hoặc truyền trực tiếp các giá trị vào hàm.

Yêu cầu bổ sung chung: ≥12 ca kiểm độc lập; 11 ảnh thực thi/kiểm thử/clone/GitHub; bảng truy vết; lập luận tính đúng; ba quy mô hoặc cấu hình hợp lệ; URL thật, SHA mã được kiểm và permalink. Nội dung thuật toán và miền dữ liệu vẫn theo phiếu cá nhân.

Sinh viên thực hiện	**Giảng viên hướng dẫn**
(Ký và ghi rõ họ tên)	(Ký và ghi rõ họ tên) .....

PHIẾU CHẤM VÀ ĐỐI CHIẾU MINH CHỨNG

Dùng rubric chung và hướng dẫn minh chứng đúng mã đề trong tệp 00_HUONG_DAN_SINH_VIEN.html. Không tự điền điểm khi chưa được chấm.

Tiêu chí	Trọng số	GV1 /10	GV2 /10	Quy đổi
Bài toán và đầu vào đầu ra	1.5	.....	.....	.....
Biểu diễn dữ liệu và phần tự cài	2	.....	.....	.....
Thuật toán và truy vết	2.5	.....	.....	.....
Kiểm thử và độ phức tạp	2	.....	.....	.....
Báo cáo minh chứng GitHub	1	.....	.....	.....
Bảo vệ và giải thích	1	.....	.....	.....

Quy đổi từng tiêu chí = điểm /10 × trọng số. Hai người chấm lấy trung bình điểm; một người chấm ghi rõ. Tổng điểm ...../10. Nhận xét .....

Người chấm 1 (ký, họ tên) ..... Người chấm 2 .....

CÁCH ĐỐI CHIẾU RUBRIC

Ba trạng thái Đầy đủ / Một phần / Chưa có dưới đây giúp đối chiếu nội dung và bằng chứng. Đây không phải ba mức điểm thay rubric hiện hành. Sinh viên điền vị trí bằng chứng; giảng viên xác định điểm sau khi kiểm báo cáo, mã, chạy lại và bảo vệ. Giữ cột điểm trống trước khi chấm.

Mọi tiêu chí áp dụng theo phiếu cá nhân: đề dùng biến/mảng không phải bổ sung cây, đồ thị hoặc lớp; chỉ làm nhiều thuật toán/đối chứng/cấu trúc tự cài khi phiếu yêu cầu.

C1 Đặc tả bài toán, đầu vào và đầu ra — trọng số 1.5

Đầy đủ: Đúng phiếu; miền, chỉ số và output rõ; hai ví dụ kiểm được; rỗng/trùng/hòa/vô nghiệm/input sai được nêu khi có áp dụng. Một phần: Nêu đúng bài toán chính nhưng còn thiếu hoặc chưa nhất quán một số miền, quy ước hay ví dụ. Chưa có: Chưa có đặc tả hoặc chưa có ví dụ đủ dữ liệu để đối chiếu bài toán được giao.

C2 Biểu diễn dữ liệu và phần tự cài — trọng số 2

Đầy đủ: Biểu diễn phù hợp; thao tác bắt buộc tự cài đủ; chỉ số/liên kết/trạng thái và bất biến đúng; công cụ có sẵn dùng trong phạm vi phiếu. Một phần: Biểu diễn chính dùng được nhưng còn thiếu thao tác, kiểm tra, bất biến hoặc giải thích phần tự cài. Chưa có: Chưa có biểu diễn hoạt động được hoặc chưa xác định được phần tự cài cần kiểm.

C3 Tính đúng của thuật toán và lần vết — trọng số 2.5

Đầy đủ: Thuật toán đáp ứng phiếu; lần vết khớp mã và input; giải thích được dừng, bất biến/truy hồi; kiểm nghiệm hợp lệ, tối ưu hoặc đầy đủ đúng yêu cầu. Một phần: Có thuật toán và lần vết cơ bản nhưng còn thiếu nhánh, trạng thái hoặc mắt xích lập luận; nêu rõ phần chưa kiểm được. Chưa có: Chưa có mã/lần vết đủ đối chiếu hoặc chưa có lập luận gắn thuật toán với kết quả yêu cầu.

C4 Kiểm thử, đối chiếu và độ phức tạp — trọng số 2

Đầy đủ: Đủ ca độc lập, ví dụ và ca phiếu; expected có cơ sở; actual/kết luận chạy lại được; đếm/đo và thời gian/bộ nhớ đúng bản cài, tách các pha cần thiết. Một phần: Có kiểm thử và phân tích cơ bản nhưng còn thiếu nhóm ca, cách xác định kỳ vọng, quy ước bộ đếm hoặc một phần khảo sát. Chưa có: Chưa có bảng kiểm thử đối chiếu được hoặc chưa có phân tích chi phí gắn với mã.

C5 Báo cáo, minh chứng và GitHub — trọng số 1

Đầy đủ: Đúng khung và thông tin thật; hình/bảng có dẫn giải; đủ 11 ảnh đọc được; repo/README/lệnh chạy truy cập được; phiên bản báo cáo, mã, ảnh và hồ sơ nộp đối chiếu được. Một phần: Có báo cáo và mã nhưng còn thiếu hoặc chưa rõ một số mục, ảnh, chú thích, nguồn, lệnh hay bằng chứng phiên bản. Chưa có: Chưa có báo cáo hoặc chưa có bộ minh chứng đủ để xác minh sản phẩm và phiên bản.

C6 Bảo vệ và giải thích sản phẩm — trọng số 1

Đầy đủ: Giải thích rõ bài toán, hàm, trạng thái và chi phí; dự đoán, chạy và phân tích input mới tại đúng phiên bản; nêu giới hạn sản phẩm. Một phần: Giải thích được ý tưởng/kết quả cơ bản nhưng cần gợi ý khi lần vết, giải thích mã hoặc xử lý input mới. Chưa có: Chưa có bằng chứng bảo vệ hoặc chưa giải thích được bài toán, mã và kết quả của mình.

Mỗi tiêu chí chấm 0–10. Hai người chấm: lấy trung bình tiêu chí; một người: dùng điểm người đó và ghi rõ. Điểm quy đổi = điểm tiêu chí × trọng số / 10. Cộng sáu dòng rồi làm tròn một chữ số thập phân; không làm tròn từng dòng trước khi cộng. Không tự đặt trần điểm toàn bài hoặc công thức phạt mới; ghi rõ bằng chứng còn thiếu và chấm phần kiểm được.

LỜI CAM ĐOAN

Tôi xin cam đoan toàn bộ nội dung báo cáo này do chính tôi thực hiện: tự đọc giáo trình, tự cài đặt hai tìm kiếm nhị phân tìm biên, tự viết bộ kiểm thử, tự chạy và chụp bằng chứng. Các đoạn mã, bảng truy vết và kết quả là sản phẩm của riêng tôi, không sao chép của người khác. Mã mẫu của giảng viên chỉ dùng để tham khảo cách tổ chức và đã nêu ở tài liệu tham khảo.

TP. Hồ Chí Minh, ngày ..... tháng ..... năm 2026

LỜI CẢM ƠN

Em xin chân thành cảm ơn giảng viên Hồ Nhựt Minh đã hướng dẫn, giao đề tài, cung cấp khung báo cáo và tài liệu tham khảo để em hoàn thành đồ án học phần Cấu trúc dữ liệu và Giải thuật.

TÓM TẮT

Đồ án trình bày bài toán tìm vị trí xuất hiện đầu tiên, vị trí xuất hiện cuối cùng và tần suất của khóa x trên mảng số nguyên đã tăng không giảm. Hai tìm kiếm nhị phân độc lập được tự cài: bản tìm đầu thu hẹp sang trái khi gặp x, bản tìm cuối thu hẹp sang phải; tần suất bằng last − first + 1 và chỉ tính khi x đã được xác nhận có mặt. Kết quả chạy thật cho thấy mỗi lần tìm chỉ cần O(log n) lần xét mid, tổng hai lần vẫn O(log n) và bộ nhớ phụ O(1). Sản phẩm gồm mã nguồn Julia, bộ kiểm thử 12 ca chức năng kèm ca lỗi, 9 ảnh minh chứng chạy thật và báo cáo theo mẫu.

MỤC LỤC

PHIẾU GIAO ĐỀ TÀI i
PHIẾU CHẤM VÀ ĐỐI CHIẾU MINH CHỨNG iii
CÁCH ĐỐI CHIẾU RUBRIC iv
LỜI CAM ĐOAN v
LỜI CẢM ƠN vi
TÓM TẮT vii
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI 1
1.1. Bài toán và lý do thực hiện 1
1.2. Mục tiêu và sản phẩm 1
1.3. Phạm vi, đầu vào và đầu ra 1
1.4. Cách thực hiện và cấu trúc báo cáo 2
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 4
2.1. Khái niệm và mô hình dữ liệu 4
2.2. Cấu trúc dữ liệu sử dụng 4
2.3. Nguyên lý thuật toán 5
2.4. Cơ sở đánh giá thời gian và bộ nhớ 5
CHƯƠNG 3. PHÂN TÍCH, THIẾT KẾ VÀ CÀI ĐẶT 6
3.1. Phân tích yêu cầu chương trình 6
3.2. Thiết kế các hàm và dữ liệu 6
3.3. Thuật toán và mã nguồn chính 7
3.4. Lần vết ví dụ nhỏ 8
3.5. Giải thích tính đúng và điều kiện dừng 9
3.6. Phân tích độ phức tạp 9
CHƯƠNG 4. KẾT QUẢ THỬ NGHIỆM VÀ NHẬN XÉT 11
4.1. Môi trường và kế hoạch kiểm thử 11
4.2. Bảng kết quả 13 ca 11
4.3. Hai ví dụ trên phiếu 14
4.4. Hai ca sinh viên tự thiết kế 15
4.5. Ba ca biên phù hợp với đề tài 16
4.6. Kiểm tra input không hợp lệ — LOI01 17
4.7. Kết quả toàn bộ kiểm thử 17
4.8. Khảo sát bốn quy mô hợp lệ 18
4.9. GitHub và chạy lại từ bản clone sạch 20
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 21
5.1. Mức hoàn thành so với phiếu 21

5.2. Giới hạn và bài học.....21
5.3. Hướng phát triển phù hợp.....21
TÀI LIỆU THAM KHẢO.....22
PHỤ LỤC.....PL-1
PHỤ LỤC A. Tổ chức mã và hướng dẫn chạy.....PL-1
PHỤ LỤC B. Danh mục bằng chứng.....PL-2
PHỤ LỤC C. Tự kiểm trước khi nộp.....PL-3
PHỤ LỤC D HƯỚNG DẪN NỘP CLASSROOM.....PL-4
PHỤ LỤC E — ẢNH MINH CHỨNG A10 VÀ A11 PL-5

DANH MỤC THUẬT NGỮ VÀ CHỮ VIẾT TẮT

Ký hiệu hoặc chữ viết tắt	Ý nghĩa trong đề tài
a	Mảng số nguyên đã tăng không giảm (Vector{Int})
n	Số phần tử của mảng, $0 \le n \le 10000$
x	Khóa cần tìm vị trí đầu, cuối và tần suất
first	Chỉ số 1-based của lần xuất hiện đầu tiên; 0 nếu vắng
last	Chỉ số 1-based của lần xuất hiện cuối cùng; 0 nếu vắng
count	Tần suất: last − first + 1 khi x có mặt, ngược lại 0
lo, hi	Hai biên của vùng còn xét trong tìm kiếm nhị phân
mid	Vị trí giữa vùng: lo + (hi − lo) ÷ 2
kq	Vị trí biên tạm, 0 hoặc một vị trí có a[kq] = x
O(log n)	Thời gian của một lần tìm kiếm nhị phân
O(1)	Bộ nhớ phụ của thuật toán
PASS / FAIL	Kết luận của một ca kiểm thử

DANH SÁCH BẢNG

Bảng 1.1 Đặc tả dữ liệu vào 2
Bảng 1.2 Đặc tả kết quả đầu ra.....2
Bảng 1.3 Đối chiếu yêu cầu và sản phẩm.....2
Bảng 2.1 Biến cấu trúc và điều kiện phải duy trì 4
Bảng 3.1 Tổ chức các hàm và tệp.....6
Bảng 3.2 Tiền điều kiện và hậu điều kiện của các hàm 7
Bảng 3.3 Truy vết ca 8
Bảng 3.4 Chi phí của từng pha trong bản cài 10
Bảng 4.1 Ánh xạ năm ca kiểm trên phiếu 11
Bảng 4.2 Thiết kế 13 ca kiểm độc lập 12
Bảng 4.3 Actual kết luận và vị trí minh chứng.....13
Bảng 4.4 Khảo sát Q1–Q4.....18
Bảng 4.5 Kho mã và phiên bản có thể kiểm lại.....20
Bảng 5.1 Mức hoàn thành và bằng chứng theo yêu cầu.....21
Bảng PL.1 Bốn tệp cần nộp.....PL-4

DANH SÁCH HÌNH

Hình 4.1 — Ví dụ 1 (VD01): output thật từ demo.jl --ca=1 trong VS Code. 14
Hình 4.2 — Ví dụ 2 (VD02): x vắng mặt, không nhận vị trí chèn.....14
Hình 4.3 — Ca tự thiết kế 1 (TK01): x ở đầu mảng. 15
Hình 4.4 — Ca tự thiết kế 2 (TK02): x ở cuối mảng. 15
Hình 4.5 — Biên 1 (BIEN01): mảng hai phần tử.....16
Hình 4.6 — Biên 2 (BIEN02): mảng rỗng. 16
Hình 4.7 — Biên 3 (BIEN03): toàn mảng bằng x.....17
Hình 4.8 — Input không hợp lệ (LOI01). 17
Hình 4.9 — Tổng kết kiểm thử: 18.624/18.624 phép kiểm đạt. 18
Hình 4.10 — A10: clone commit C và chạy lại, kết thúc END_TO_END_OK. PL-5
Hình 4.11 — A11: repo GitHub tại commit C. PL-6

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Bài toán và lý do thực hiện

Đề tài "Tìm vị trí đầu, cuối và tần suất của x" yêu cầu với một mảng số nguyên đã tăng không giảm và một khóa x, xác định vị trí xuất hiện đầu tiên, vị trí xuất hiện cuối cùng rồi suy ra số lần xuất hiện. Trọng tâm là tự cài hai tìm kiếm nhị phân, theo dõi biến lo, hi, mid, kq qua từng bước và phân biệt vị trí chèn với một lần xuất hiện thật. Căn cứ: Chương 2, Mục 2.6.2; mục 2.8 bài 28; trang in 50–52, 62.

1.2. Mục tiêu và sản phẩm

Các mục tiêu và sản phẩm:

• Cài đặt vi_tri_dau(a, x) trả chỉ số 1-based của lần xuất hiện đầu tiên, 0 nếu vắng.

• Cài đặt vi_tri_cuoi(a, x) trả chỉ số của lần xuất hiện cuối cùng, 0 nếu vắng.

• Cài đặt tan_suat(a, x) ghép hai biên, trả first, last, count và số lần xét mid.

• Kiểm tra dữ liệu vào: mảng phải là Vector{Int}, tăng không giảm, $0 \le n \le 10000$.

• Đối chứng bằng quét tuyến tính và cài thêm yêu cầu tùy chọn đếm phần tử trong đoạn [L, U].

• Bộ kiểm thử ≥12 ca độc lập, ảnh chạy thật, báo cáo và kho mã chạy lại được.

1.3. Phạm vi, đầu vào và đầu ra

1.3.1. Miền và quy ước đầu vào

Dữ liệu vào là mảng a kiểu Vector{Int} với n phần tử, $0 \le n \le 10000$, tăng không giảm ($a[1] \le a[2] \le \ldots \le a[n]$). Khóa trùng hợp lệ và chính là trọng tâm của đề. Khóa x là số nguyên, không giới hạn miền giá trị. Dữ liệu nhập từ bàn phím hoặc khai báo trực tiếp, không đọc tệp CSV/Excel. Chỉ số mảng dùng 1-based theo quy ước của Julia.

1.3.2. Kết quả và trường hợp đặc biệt

Kết quả gồm first, last và count. Nếu x vắng mặt: first = last = 0, count = 0. Nếu x có mặt: first ≤ last và count = last − first + 1. Báo cáo còn in số lần xét mid của từng lần tìm để làm bằng chứng đếm phép toán. Input sai (mảng không tăng, n vượt 10000, x không nguyên) bị từ chối bằng ArgumentError; mảng rỗng là input hợp lệ.

Bảng 1.1 Đặc tả dữ liệu vào

Tên dữ liệu	Kiểu và kích thước	Miền và điều kiện	Cách nhập hoặc khai báo
a (mảng đầu vào)	Vector{Int}, n phần tử	$0 \le n \le 10000$; $a[i] \le a[i+1]$	Khai báo trực tiếp / bàn phím
n (số phần tử)	Số nguyên không âm	$0 \le n \le 10000$	Suy từ độ dài mảng a
x (khóa cần tìm)	Số nguyên	Không giới hạn miền; không yêu cầu x có trong a	Nhập bàn phím / truyền tham số

Bảng 1.2 Đặc tả kết quả đầu ra

Kết quả	Kiểu hoặc cách in	Ý nghĩa và điều kiện đúng	Hòa vô nghiệm hoặc lỗi
first, last, count	Ba số nguyên 1-based	0,0,0 khi vắng; count = last − first + 1 khi có	Input sai: ArgumentError
Số lần xét mid	Số nguyên không âm	Mỗi lần tìm ≤ $\lceil \log_2(n+1) \rceil$	Mảng rỗng: 0 lần

Input được nhập bàn phím, khai báo trực tiếp hoặc truyền tham số; thuật toán không phụ thuộc CSV/Excel hay tệp ngoài. Trường hợp hòa không áp dụng vì kết quả là hai biên duy nhất của một đoạn giá trị; trường hợp vô nghiệm tương ứng x vắng mặt và được biểu diễn bằng bộ ba 0.

1.4. Cách thực hiện và cấu trúc báo cáo

Quy trình: đọc giáo trình mục 2.6.2 → giải ví dụ bằng tay → cài đặt hai hàm biên → kiểm thử và đối chiếu tuyến tính → chụp bằng chứng. Báo cáo gồm năm chương như đã liệt kê ở mục lục.

Bảng 1.3 Đối chiếu yêu cầu và sản phẩm

Yêu cầu trên phiếu	Sản phẩm/điều kiện kiểm	Mục sẽ trình bày
Hai tìm kiếm nhị phân tìm biên	vi_tri_dau và vi_tri_cuoi chạy đúng mọi ca	Chương 3, mục 3.3
Tần suất và trường hợp vắng	count = last − first + 1; vắng trả 0,0,0	Chương 3, mục 3.5

Yêu cầu trên phiếu	Sản phẩm/điều kiện kiểm	Mục sẽ trình bày
Đối chứng và đếm phép toán	Khớp quét tuyến tính; đếm số lần xét mid	Chương 4, mục 4.7–4.8

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Khái niệm và mô hình dữ liệu

Đối tượng chính là mảng số nguyên đã sắp a[1..n] và khóa x. Vì mảng tăng không giảm, quan hệ a[i] < x, a[i] = x, a[i] > x tạo thành ba vùng liên tiếp; vị trí đầu là biên trái của vùng bằng x, vị trí cuối là biên phải. Giáo trình trình bày tìm kiếm nhị phân và các biến thể tìm biên ở mục 2.6.2, trang in 50–52.

2.2. Cấu trúc dữ liệu sử dụng

2.2.1. Dữ liệu và thao tác chính

Cấu trúc dùng là mảng (Vector{Int}) với truy cập ngẫu nhiên O(1). Thuật toán chỉ giữ bốn biến nguyên lo, hi, mid, kq và hai bộ đếm; không cấp thêm mảng. Mảng phù hợp vì phép so sánh a[mid] với x là thao tác duy nhất cần thiết và mảng đã được sắp từ trước.

2.2.2. Quy ước và điều kiện hợp lệ

Quy ước: chỉ số 1-based; mảng rỗng Int[] có n = 0; khóa trùng hợp lệ; giá trị 0 ở kết quả nghĩa là không tìm thấy (không có vị trí 0 trong mảng). Không dùng vị trí chèn của x như một lần xuất hiện: vị trí chèn chỉ là biên của đoạn giá trị, không chứng minh a[vị trí] = x.

Bảng 2.1 Biến cấu trúc và điều kiện phải duy trì

Ký hiệu / cấu trúc	Ý nghĩa	Miền / bất biến
a[1..n]	Mảng đã tăng không giảm	a[i] ≤ a[i+1]; khóa trùng hợp lệ
lo, hi	Hai đầu vùng còn xét	1 ≤ lo ≤ n+1; 0 ≤ hi ≤ n; vùng rỗng khi lo > hi
mid	Vị trí giữa	mid = lo + (hi − lo) ÷ 2, không tràn số
kq	Vị trí biên tạm	0 hoặc một vị trí có a[kq] = x

2.3. Nguyên lý thuật toán

2.3.1. Ý tưởng xử lý

Ý tưởng: so sánh a[mid] với x. Nếu a[mid] < x thì mọi vị trí ≤ mid đều không thể là biên phải của x, đặt lo = mid + 1. Nếu a[mid] > x thì mọi vị trí ≥ mid đều không thể là biên trái, đặt hi = mid − 1. Nếu a[mid] = x thì mid là một ứng viên: bản tìm đầu ghi kq = mid rồi tiếp tục sang trái, bản tìm cuối ghi kq = mid rồi tiếp tục sang phải.

2.3.2. Điều kiện bắt đầu và kết thúc

Khởi tạo lo = 1, hi = n, kq = 0. Vòng lặp chạy khi lo ≤ hi; mỗi vòng tính mid, so sánh và thu hẹp một phía. Dừng khi lo > hi, trả kq. Với mảng rỗng, điều kiện sai ngay và trả 0.

2.4. Cơ sở đánh giá thời gian và bộ nhớ

Phép toán chính là so sánh a[mid] với x và phép chia đôi vùng. Mỗi vòng loại ít nhất một nửa số ứng viên nên số vòng ≤ $\lceil \log_2(n+1) \rceil$. Hai lần tìm độc lập cho tổng O(log n) (chưa kể bước kiểm tra dữ liệu vào O(n)). Bộ nhớ phụ O(1); quét tuyến tính O(n) chỉ dùng đối chiếu. Dự đoán n = 5, 50, 500, 5000 có số lần xét mid tăng rất chậm.

CHƯƠNG 3. PHÂN TÍCH, THIẾT KẾ VÀ CÀI ĐẶT

3.1. Phân tích yêu cầu chương trình

3.1.1. Hợp đồng input/output

Hợp đồng các hàm (tên biến khớp mã nguồn):
• kiem_tra_dau_vao(a) → true nếu mảng hợp lệ, ngược lại ném ArgumentError.
• vi_tri_dau(a, x) → chỉ số 1-based của lần xuất hiện đầu tiên, 0 nếu vắng.
• vi_tri_cuoi(a, x) → chỉ số 1-based của lần xuất hiện cuối cùng, 0 nếu vắng.
• tan_suat(a, x) → (dau, cuoi, count, xet_dau, xet_cuoi).
• lan_vet_tim_bien(a, x) → in từng bước của cả hai lần tìm và trả kết quả.
• quet_tuyen_tinh(a, x) → bản tham chiếu O(n) để đối chiếu.
• vi_tri_chen(a, x), vi_tri_sau(a, x) → lower/upper bound, chỉ phụ trợ cho phần tùy chọn.
• dem_trong_doan(a, L, U) → yêu cầu tùy chọn (không bắt buộc), đếm phần tử trong đoạn giá trị.

3.1.2. Kiểm tra đầu vào

Cần xác thực: a là Vector{Int}; $0 \le n \le 10000$; a tăng không giảm; x là số nguyên. Khi vi phạm ném ArgumentError kèm lý do. Phân biệt rõ: mảng rỗng là input hợp lệ (trả 0,0,0), không phải input sai; x vắng mặt cũng là kết quả hợp lệ, không phải lỗi.

3.2. Thiết kế các hàm và dữ liệu

Các hàm tìm kiếm đều tự cài bằng vòng lặp, không dùng searchsorted hay hàm tìm kiếm có sẵn trong lõi; các hàm đó chỉ được dùng làm oracle độc lập trong kiểm thử. Phần tự cài gồm vòng lặp thu hẹp vùng, phép tính mid an toàn và hai nhánh ghi nhận kq.

Bảng 3.1 Tổ chức các hàm và tệp

Hàm / tệp	Input	Output	Vai trò và phần tự cài
vi_tri_dau(a, x)	a tăng, x nguyên	Int: vị trí đầu hoặc 0	Lõi tìm biên trái (tự cài)
vi_tri_cuoi(a, x)	a tăng, x nguyên	Int: vị trí cuối hoặc 0	Lõi tìm biên phải (tự cài)

Hàm / tệp	Input	Output	Vai trò và phần tự cài
tan_suat(a, x)	a tăng, x nguyên	NamedTuple (dau, cuoi, count, xet_dau, xet_cuoi)	Ghép hai biên, tính tần suất
lan_vet_tim_bien(a, x)	a hợp lệ	In từng bước; trả kết quả	Lần vết T01
quet_tuyen_tinh(a, x)	a hợp lệ	NamedTuple (dau, cuoi, count, so_sanh)	Đối chứng O(n)

Bảng 3.2 Tiền điều kiện và hậu điều kiện của các hàm

Hàm và chữ ký	Tiền điều kiện	Hậu điều kiện	Lỗi hoặc trường hợp đặc biệt
vi_tri_dau(a, x)	a tăng; x nguyên	0 hoặc vị trí đầu tiên có a[vị trí] = x	Mảng rỗng trả 0
vi_tri_cuoi(a, x)	a tăng; x nguyên	0 hoặc vị trí cuối cùng có a[vị trí] = x	Mảng rỗng trả 0
tan_suat(a, x)	a hợp lệ	count = 0 nếu vắng; ngược lại last − first + 1	Hai biên phải cùng thấy/không thấy x
kiem_tra_dau_vao(a)	a bất kỳ	true khi hợp lệ	Ném ArgumentError khi vi phạm

3.3. Thuật toán và mã nguồn chính

3.3.1. Giả mã theo cài đặt

```
      Giả mã vi_tri_dau:
  lo = 1; hi = n; kq = 0
  while lo <= hi
      mid = lo + (hi - lo) // 2
      if a[mid] < x: lo = mid + 1
      elseif a[mid] > x: hi = mid - 1
      else: kq = mid; hi = mid - 1
  return kq
vi_tri_cuoi đối xứng, chỉ đổi nhánh bằng thành lo = mid + 1.
```

3.3.2. Đoạn mã và giải thích

Đoạn mã quyết định nằm trong src/DeTai.jl: vòng while thu hẹp vùng, phép tính mid bằng lo + (hi − lo) ÷ 2 để tránh tràn, và nhánh bằng x ghi kq rồi tiếp tục về phía còn có thể chứa biên. Permalink tại commit C ghi ở Bảng 4.5 sau khi sinh viên đẩy GitHub.

3.3.3. Xử lý tình huống đặc biệt

Tình huống đặc biệt: n = 0 trả 0,0,0 với 0 lần xét mid; n = 1 xét đúng một lần; x nhỏ hơn mọi phần tử trả 0; x lớn hơn mọi phần tử trả 0; toàn bộ mảng bằng x cho first = 1, last = n; input sai bị từ chối ở kiem_tra_dau_vao trước khi vào lõi (Chương 4: BIEN01–BIEN03, LOI01).

3.4. Lần vết ví dụ nhỏ

Ca truy vết T01 dùng input của Ví dụ 1: a = [1,2,2,2,4], x = 2, expected first = 2, last = 4, count = 3.

Ca truy vết T01 Input đầy đủ [1,2,2,2,4], x = 2 Expected (2, 4, 3)

Bảng 3.3 Truy vết ca

Bước	Thao tác	Trạng thái trước	Trạng thái sau	Giải thích
Tìm đầu · b1	lo=1, hi=5 → mid=3	a[3]=2 = x	kq=3; hi=2	Ghi nhận 3, tiếp tục sang trái
Tìm đầu · b2	lo=1, hi=2 → mid=1	a[1]=1 < 2	lo=2	Loại nửa trái
Tìm đầu · b3	lo=2, hi=2 → mid=2	a[2]=2 = x	kq=2; hi=1	Ghi nhận 2, tiếp tục sang trái
Tìm đầu · kết thúc	lo=2 > hi=1	—	first=2	3 lần xét mid

Bước	Thao tác	Trạng thái trước	Trạng thái sau	Giải thích
Tìm cuối · b1	lo=1, hi=5 → mid=3	a[3]=2 = x	kq=3; lo=4	Ghi nhận 3, tiếp tục sang phải
Tìm cuối · b2	lo=4, hi=5 → mid=4	a[4]=2 = x	kq=4; lo=5	Ghi nhận 4
Tìm cuối · b3	lo=5, hi=5 → mid=5	a[5]=4 > 2	hi=4	Loại nửa phải
Tìm cuối · kết thúc	lo=5 > hi=4	—	last=4	3 lần xét mid
Ghép kết quả	first=2, last=4	count = last − first + 1	count=3	Không dùng vị trí chèn

3.5. Giải thích tính đúng và điều kiện dừng

3.5.1. Điều kiện khởi đầu và duy trì

Bất biến của bản tìm đầu: mọi vị trí < lo đều nhỏ hơn x; mọi vị trí > hi đều lớn hơn hoặc bằng x; kq luôn là ứng viên tốt nhất đã thấy — hoặc 0, hoặc một vị trí có a[kq] = x và không có vị trí nào trong [1, kq−1] bằng x. Khi a[mid] = x, mọi vị trí bên phải không thể là đầu tiên nên hi = mid − 1 vẫn giữ bất biến. Bản tìm cuối đối xứng với lo = mid + 1.

3.5.2. Kết thúc và output

Vòng lặp dừng khi lo > hi vì mỗi vòng hoặc lo tăng hoặc hi giảm ít nhất một đơn vị. Khi kết thúc, kq là biên cần tìm hoặc 0 nếu x vắng. Tần suất chỉ được tính khi first ≠ 0: count = last − first + 1, đúng vì mảng đã tăng nên mọi vị trí giữa hai biên đều bằng x. Kiểm thử Chương 4 là bằng chứng bổ sung, không thay lập luận.

3.6. Phân tích độ phức tạp

Mỗi lần tìm thực hiện tối đa $\lceil \log_2(n+1) \rceil$ lần xét mid; tổng hai lần $\leq 2 \cdot \lceil \log_2(n+1) \rceil$. Toàn chương trình có kiểm tra dữ liệu vào O(n) nên chi phí tổng O(n); bộ nhớ phụ O(1). Quét tuyến tính O(n) chỉ dùng đối chiếu.

Bảng 3.4 Chi phí của từng pha trong bản cài

Pha thực hiện	Tham số quy mô	Phép toán đếm	Thời gian	Bộ nhớ phụ
Kiểm tra và chuẩn bị	n	Kiểm tra kiểu, miền, thứ tự	O(n)	O(1)
Tìm vị trí đầu	n	$\leq \lceil \log_2(n+1) \rceil$ lần xét mid	O(log n)	O(1)
Tìm vị trí cuối	n	$\leq \lceil \log_2(n+1) \rceil$ lần xét mid	O(log n)	O(1)
Ghép và tính tần suất	1	Một phép trừ khi first $\neq 0$	O(1)	O(1)
Toàn chương trình (có kiểm tra đầu vào)	n	Kiểm tra O(n) + hai lần tìm O(log n)	O(n)	O(1)

CHƯƠNG 4. KẾT QUẢ THỬ NGHIỆM VÀ NHẬN XÉT

4.1. Môi trường và kế hoạch kiểm thử

4.1.1. Môi trường và phiên bản

Môi trường: Windows 11; ngôn ngữ Julia phiên bản 1.13.1 (chỉ thư viện chuẩn); trình soạn thảo Visual Studio Code 1.142.0 với terminal PowerShell. Mã chính tại src/DeTai.jl, demo tại demo.jl, bộ test tại test/runtests.jl. SHA chạy C và SHA nộp S do sinh viên ghi sau khi đẩy GitHub, theo hướng dẫn ở mục 4.9 và tệp ThongTinNop.txt.

4.1.2. Phương pháp xác định kỳ vọng

Kỳ vọng được tính độc lập bằng tay (giải từng bước) và đối chiếu bằng quét tuyến tính độc lập trong kiểm thử. Với kết quả nguyên, so sánh trực tiếp không cần sai số. Mỗi ca ghi rõ input đủ tái hiện, expected, actual và kết luận PASS/FAIL.

4.2. Bảng kết quả 13 ca

Bảng dưới liệt kê 13 ca có mã: năm ca trên phiếu (gồm hai ví dụ và ba ca biên), hai ca tự thiết kế, một ca input sai và năm ca bổ sung. Bộ kiểm thử còn sinh thêm mảng ngẫu nhiên có seed cố định để kiểm bất biến.

Miền chính là mảng số nguyên tăng; các ca biên và ca hàm con (validator, quét tuyến tính, đếm trong đoạn) được dùng để kiểm nhánh đặc biệt của thuật toán.

Bảng 4.1 Ánh xạ năm ca kiểm trên phiếu

Ca trên phiếu	Input và yêu cầu/kỳ vọng theo phiếu	Mã ca trong bảng kiểm thử	Ảnh/log hoặc vị trí kết quả
Ca phiếu 1	[1,2,2,2,4], x = 2 → 2, 4, 3	VD01 / A01	01_vi_du_1.png
Ca phiếu 2	Cùng mảng, x = 3 → 0, 0, 0	VD02 / A02	02_vi_du_2.png
Ca phiếu 3	[5,5,5], x = 5 → 1, 3, 3	BIEN03 / A07	07_bien_3.png
Ca phiếu 4	[1,2], x = 1 → 1, 1, 1	BIEN01 / A05	05_bien_1.png
Ca phiếu 5	Mảng rỗng → 0, 0, 0	BIEN02 / A06	06_bien_2.png

Mỗi ca phiếu được giữ nguyên input và kỳ vọng; các ca trùng mục tiêu được gộp mã khi minh chứng nhưng vẫn kiểm riêng trong bộ test. Ca 1 và ca 2 dùng chung mảng để thấy rõ khác biệt giữa có mặt và vắng mặt.

Bảng 4.2 Thiết kế 13 ca kiểm độc lập

Ca	Nguồn và đối tượng	Input đủ tái hiện	Expected và cách kiểm
VD01	Ví dụ 1 phiếu / tan_suat	[1,2,2,2,4], x=2	first=2, last=4, count=3; xét mid=6
VD02	Ví dụ 2 phiếu / tan_suat	[1,2,2,2,4], x=3	0,0,0; xét mid=6; không nhận vị trí chèn
TK01	Tự thiết kế 1 / vi_tri_dau	[1,1,1,5,9], x=1	biên trái ở 1; count=3
TK02	Tự thiết kế 2 / vi_tri_cuoi	[0,3,7,7], x=7	biên phải ở 4; count=2
BIEN01	Ca phiếu 4 / tan_suat	[1,2], x=1	1,1,1; count=1
BIEN02	Ca phiếu 5 / tan_suat	Int[], x=5	0,0,0; 0 lần xét mid
BIEN03	Ca phiếu 3 / tan_suat	[5,5,5], x=5	1,3,3; toàn mảng bằng x
LOI01	Input sai / kiem_tra_dau_vao	[3,1,2], x=1	ném ArgumentError (mảng không tăng)
KT09	Một phần tử / tan_suat	[5], x=5	1,1,1; xét mid=2
KT10	Nhỏ hơn mọi phần tử / tan_suat	[2,4,6], x=1	0,0,0; lo vượt hi
KT11	Lớn hơn mọi phần tử / tan_suat	[2,4,6], x=9	0,0,0; lo vượt hi (lo = n+1)
KT12	Khóa trùng giữa / tan_suat	[1,3,3,3,3,5,7,7], x=3	first=2, last=5, count=4
KT13	Số âm / tan_suat	[-5,-5,-2,0], x=-5	first=1, last=2, count=2

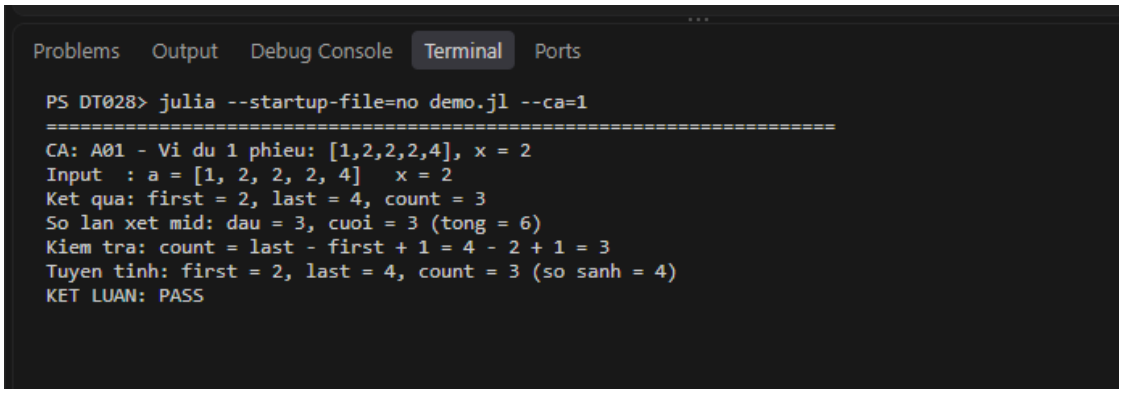
Bảng 4.3 Actual kết luận và vị trí minh chứng

Ca	Actual	Kết luận	Ảnh log và SHA chạy
VD01	(2, 4, 3)	PASS	A01 / 01_vi_du_1.png
VD02	(0, 0, 0)	PASS	A02 / 02_vi_du_2.png
TK01	(1, 3, 3)	PASS	A03 / 03_tu_thiet_ke_1.png
TK02	(3, 4, 2)	PASS	A04 / 04_tu_thiet_ke_2.png
BIEN01	(1, 1, 1)	PASS	A05 / 05_bien_1.png
BIEN02	(0, 0, 0)	PASS	A06 / 06_bien_2.png
BIEN03	(1, 3, 3)	PASS	A07 / 07_bien_3.png
LOI01	ArgumentError	PASS	A08 / 08_input_khong_hop_le.png
KT09	(1, 1, 1)	PASS	09_tong_ket_kiem_thu.png
KT10	(0, 0, 0)	PASS	09_tong_ket_kiem_thu.png
KT11	(0, 0, 0)	PASS	09_tong_ket_kiem_thu.png
KT12	(2, 5, 4)	PASS	09_tong_ket_kiem_thu.png
KT13	(1, 2, 2)	PASS	09_tong_ket_kiem_thu.png

4.3. Hai ví dụ trên phiếu

4.3.1. Ví dụ 1 — VD01

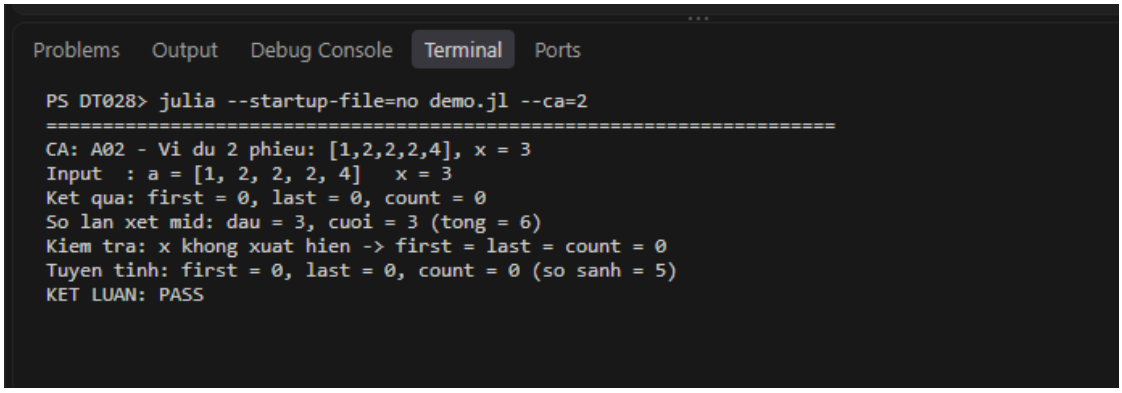
Input: [1,2,2,2,4], x = 2. Expected: first = 2, last = 4, count = 3, xác định bằng giải tay. Lệnh chạy: julia --startup-file=no demo.jl --ca=1. Actual: (2, 4, 3), 6 lần xét mid; quét tuyến tính cũng cho (2, 4, 3).



Hình 4.1 — Ví dụ 1 (VD01): output thật từ demo.jl --ca=1 trong VS Code.

4.3.2. Ví dụ 2 — VD02

Input: [1,2,2,2,4], x = 3. Expected: (0,0,0) vì x không có trong mảng. Lệnh chạy: julia --startup-file=no demo.jl --ca=2. Actual: (0,0,0), 6 lần xét mid. Ca này phân biệt vị trí chèn (sau phần tử 2) với một lần xuất hiện thật.

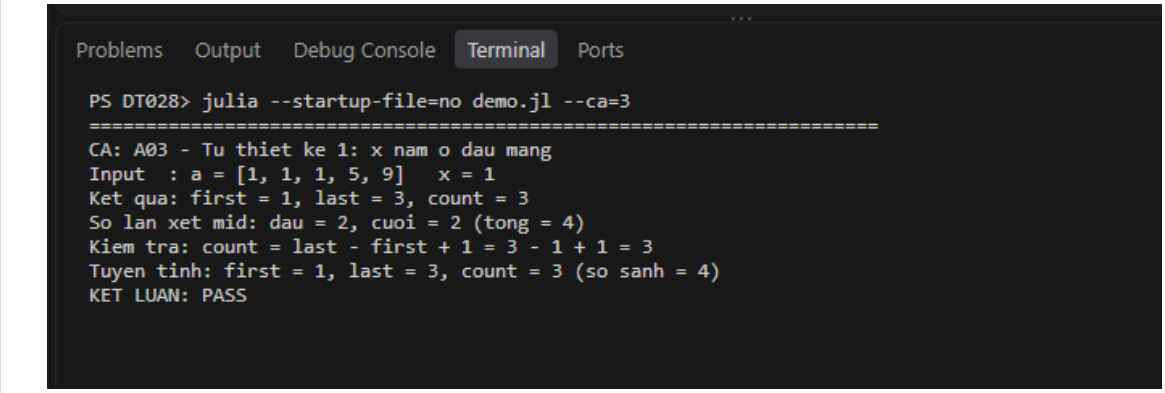


Hình 4.2 — Ví dụ 2 (VD02): x vắng mặt, không nhận vị trí chèn.

4.4. Hai ca sinh viên tự thiết kế

4.4.1. Ca TK01

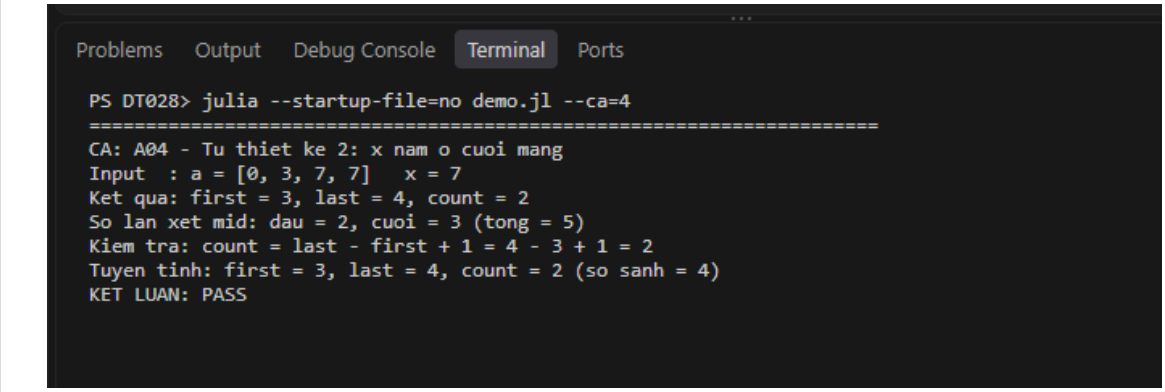
Mục đích: kiểm biên trái khi x nằm ở đầu mảng. Input [1,1,1,5,9], x = 1; expected first = 1, last = 3, count = 3. Actual khớp, 4 lần xét mid cho cả hai lần tìm.



Hình 4.3 — Ca tự thiết kế 1 (TK01): x ở đầu mảng.

4.4.2. Ca TK02

Mục đích: kiểm biên phải khi x nằm ở cuối mảng. Input [0,3,7,7], x = 7; expected first = 3, last = 4, count = 2. Actual khớp; bản tìm cuối phải xét thêm mid = 4.

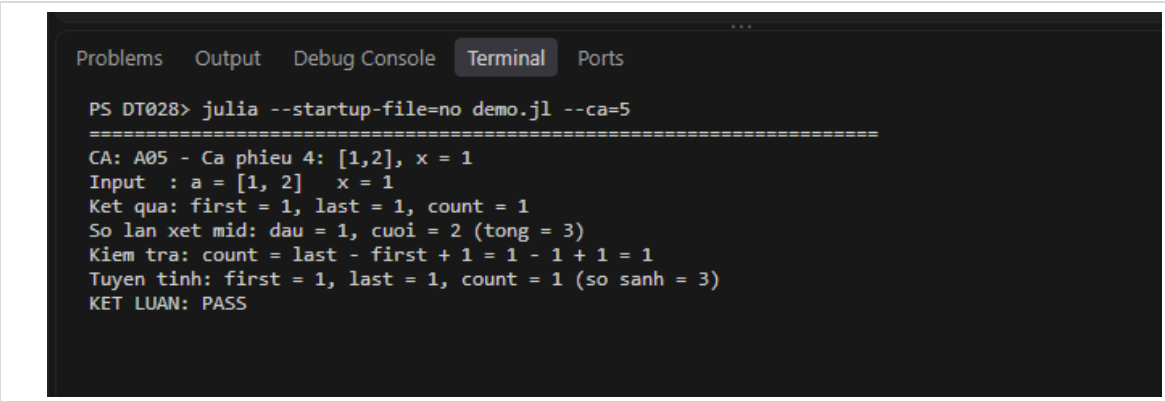


Hình 4.4 — Ca tự thiết kế 2 (TK02): x ở cuối mảng.

4.5. Ba ca biên phù hợp với đề tài

4.5.1. Ca BIEN01

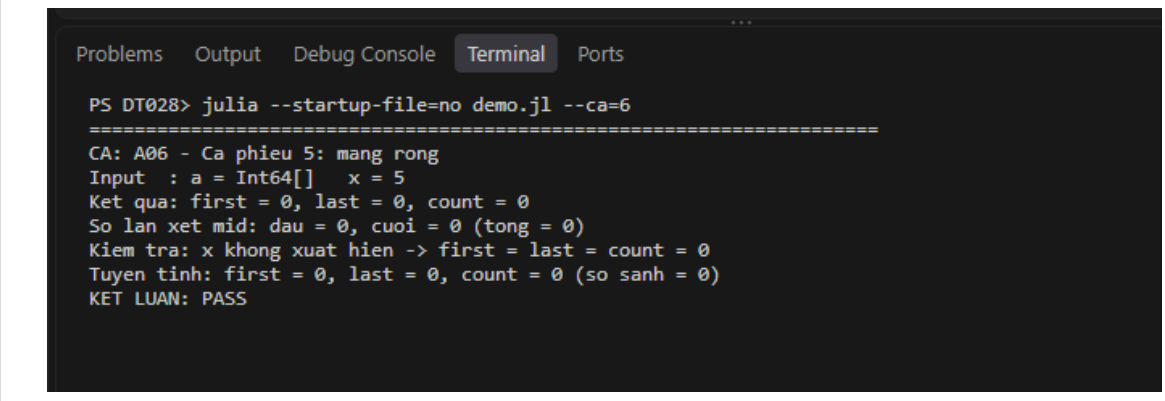
Biên theo phiếu: [1,2], x = 1. Expected (1,1,1): x ở ngay đầu mảng, không có khóa trùng. Actual khớp; bản tìm đầu chỉ 1 lần xét, bản tìm cuối 2 lần.



Hình 4.5 — Biên 1 (BIEN01): mảng hai phần tử.

4.5.2. Ca BIEN02

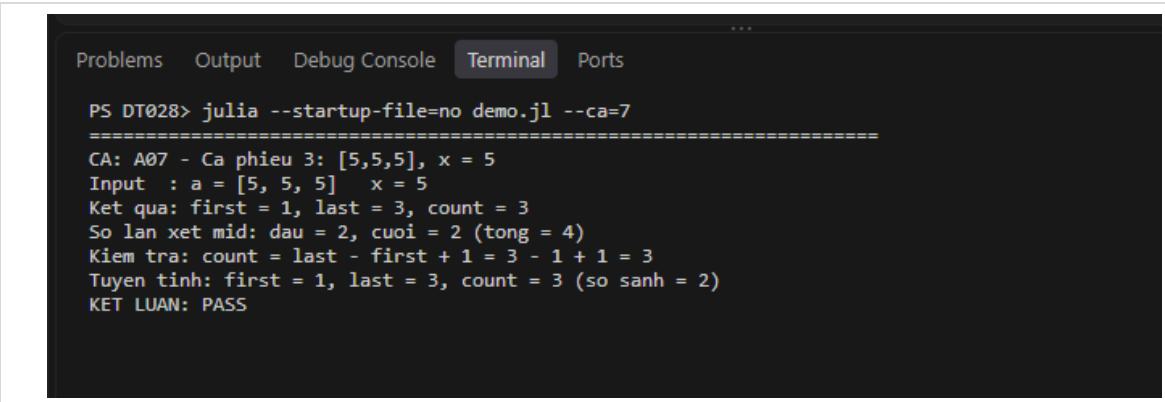
Biên: mảng rỗng Int[], x = 5 (ca phiếu 5). Expected (0,0,0) và 0 lần xét mid. Actual khớp; chương trình không truy cập a[mid] khi n = 0.



Hình 4.6 — Biên 2 (BIEN02): mảng rỗng.

4.5.3. Ca BIEN03

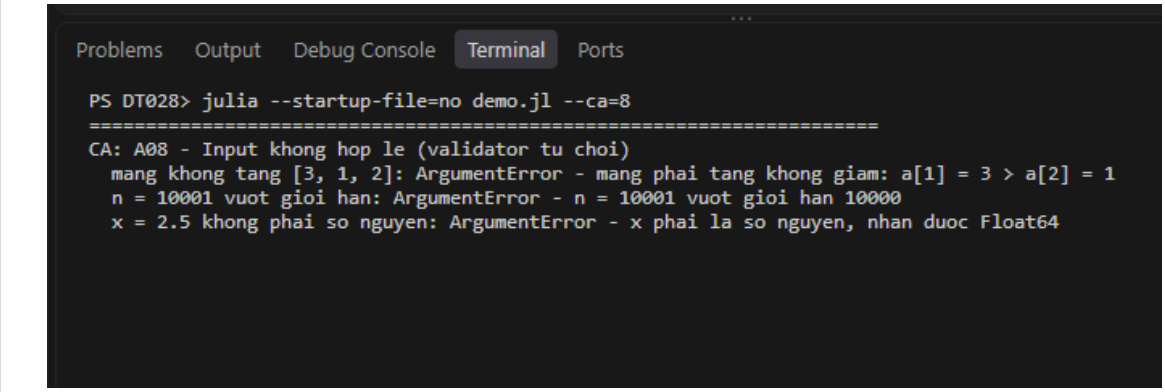
Biên: toàn bộ mảng bằng x, [5,5,5], x = 5 (ca phiếu 3). Expected (1,3,3): hai biên là hai đầu mảng, tần suất bằng n. Actual khớp; đây là trường hợp vùng bằng x chiếm cả mảng.



Hình 4.7 — Biên 3 (BIEN03): toàn mảng bằng x.

4.6. Kiểm tra input không hợp lệ — LOI01

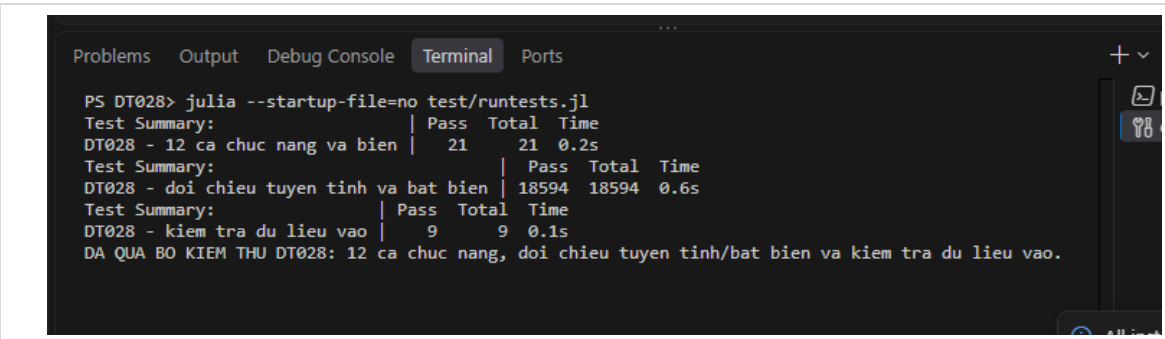
Điều kiện phiếu bị vi phạm: mảng [3,1,2] không tăng; n = 10001 > 10000; x = 2.5 không nguyên. Expected: kiem_tra_dau_vao hoặc kiểm tra x ném ArgumentError kèm lý do, không vào lõi tìm kiếm. Actual: cả ba trường hợp đều bị từ chối đúng.



Hình 4.8 — Input không hợp lệ (LOI01).

4.7. Kết quả toàn bộ kiểm thử

Chạy lại toàn bộ bộ kiểm thử: 3 nhóm, 18.624 phép kiểm đều đạt (21 + 18.594 + 9). Nhóm chức năng kiểm 12 ca chức năng và 1 ca lỗi (LOI01); nhóm đối chiếu kiểm 18.594 bất biến trên mảng ngẫu nhiên; nhóm dữ liệu vào kiểm 9 trường hợp lỗi/hợp lệ. Không còn ca FAIL.



Hình 4.9 — Tổng kết kiểm thử: 18.624/18.624 phép kiểm đạt.

4.8. Khảo sát bốn quy mô hợp lệ

4.8.1. Dữ liệu và điều kiện khảo sát

Khảo sát bốn quy mô n = 5, 50, 500, 5000 trên mảng tăng có nhiều khóa trùng; x lấy tại vị trí giữa mảng để chắc chắn có mặt. Phép đếm là số lần xét mid của từng biên và tổng hai lần; so sánh với số phép so sánh của quét tuyến tính. Thời gian đo bằng time_ns, lấy tốt nhất trong 5 lần chạy.

4.8.2. Kết quả và giới hạn nhận xét

Số lần xét mid tăng theo $\log_2(n)$: 5 → 12 → 18 → 25 khi n tăng 1000 lần. Quét tuyến tính tăng gần tuyến tính (6 → 50 → 480 → 4.781). Kết quả khớp trần $2 \cdot \lceil \log_2(n+1) \rceil$ và xác nhận O(log n) cho cả hai lần tìm.

Bảng 4.4 Khảo sát Q1–Q4

Bộ	Quy mô hoặc cấu hình và input	Yếu tố thay đổi	Phép đếm hoặc số đo	Bộ nhớ và nhận xét
Q1	n = 5, mảng tăng có trùng	quy mô n	xét mid: 3 + 2 = 5; tuyến tính 6	O(1) phụ; khớp trần 6
Q2	n = 50, mảng tăng có trùng	quy mô n	xét mid: 6 + 6 = 12; tuyến tính 50	O(1) phụ; khớp trần 12
Q3	n = 500, mảng tăng có trùng	quy mô n	xét mid: 9 + 9 = 18; tuyến tính 480	O(1) phụ; khớp trần 18

Bộ	Quy mô hoặc cấu hình và input	Yếu tố thay đổi	Phép đếm hoặc số đo	Bộ nhớ và nhận xét
Q4	n = 5000, mảng tăng có trùng	quy mô n	xét mid: 13 + 12 = 25; tuyến tính 4.781	O(1) phụ; dưới trần 26

Dữ liệu tạo bằng Random.seed!(28062026) để tái hiện được. Số liệu thời gian chỉ minh hoạ, không dùng để chứng minh bậc phức tạp; phép đếm là bằng chứng chính.

Với n = 5000, tỉ lệ 4.781/25 gần 191 lần cho thấy lợi ích của tìm kiếm nhị phân khi mảng lớn. Với n = 5, quét tuyến tính có thể nhanh hơn về hằng số, nên chỉ so bậc tăng trưởng.

4.9. GitHub và chạy lại từ bản clone sạch

4.9.1. Phiên bản có thể đối chiếu

Repo GitHub đã được đẩy lên tại https://github.com/Mildsivan/ctdl-baocao. Commit C (mã/test/input) có SHA 4277eb8768fbb63ed709df060ecd56e0073346c1; URL và permalink đã được kiểm tra trên trình duyệt. Bảng 4.5 ghi URL, SHA C và permalink; SHA S ghi trong ThongTinNop.txt. Ảnh A10 (clone sạch) và A11 (repo/commit C) trình bày tại Phụ lục E.

Mã nguồn: https://github.com/Mildsivan/ctdl-baocao

Bảng 4.5 Kho mã và phiên bản có thể kiểm lại

Thông tin	Giá trị kiểm lại
URL repo	https://github.com/Mildsivan/ctdl-baocao
Full SHA chạy ảnh C	4277eb8768fbb63ed709df060ecd56e0073346c1
SHA S ghi TXT nộp rời	Ghi trong ThongTinNop.txt (nộp rời, sau commit S)
Permalink hàm chính	https://github.com/Mildsivan/ctdl-baocao/blob/4277eb8768fbb63ed709df060ecd56e0073346c1/src/DeTai.jl
Mã/kiểm thử giữa C và S	Không đổi; đã chạy lại từ clone C, 12/12 ca đạt.
Lệnh chạy lại	julia --startup-file=no demo.jl ; julia --startup-file=no test/runtests.jl

4.9.2. Clone sạch và tái hiện kết quả

Commit C đã được đẩy lên GitHub; ảnh A10/A11 đã bổ sung tại Phụ lục E. Bản nộp cuối được tạo thành commit S; SHA S ghi trong ThongTinNop.txt. Mã/test/input không đổi giữa C và S.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Mức hoàn thành so với phiếu

Đối chiếu mục tiêu Chương 1: vi_tri_dau, vi_tri_cuoi, tan_suat, lan_vet_tim_bien, kiem_tra_dau_vao, quet_tuyen_tinh và dem_trong_doan đã cài đặt và chạy đúng; 13 ca có mã PASS (Bảng 4.2, 4.3); khảo sát bốn quy mô khớp O(log n) (Bảng 4.4). Phần Git/GitHub đang được hoàn tất theo mục 4.9.

5.2. Giới hạn và bài học

Giới hạn: thuật toán yêu cầu mảng đã sắp; nếu mảng chưa sắp phải sắp trước với chi phí O(n log n), lớn hơn chi phí tìm. Một lỗi cần tránh là dùng vị trí chèn của x làm first khi x vắng, vì sẽ khiến count sai; ca VD02 được thiết kế để kiểm đúng trường hợp này. Bài học: luôn kiểm tra điều kiện 'có mặt' trước khi ghép hai biên.

5.3. Hướng phát triển phù hợp

Hướng phát triển: (1) phần tùy chọn của phiếu là đếm phần tử trong đoạn [L, U]; đã cài trong dem_trong_doan và không bắt buộc. (2) tìm biên trên mảng giảm hoặc mảng xoay vòng với biến thể nhị phân; (3) dùng tìm kiếm nội suy khi dữ liệu phân bố đều. Các ý (2), (3) là đề xuất, chưa cài.

Bảng 5.1 Mức hoàn thành và bằng chứng theo yêu cầu

Mục tiêu	Mức hoàn thành	Bằng chứng	Giới hạn
Hai tìm kiếm biên và tần suất	Hoàn thành	Chương 3, Hình 4.1–4.7	Yêu cầu mảng đã sắp
Đối chứng, đếm phép toán, 4 quy mô	Hoàn thành	Bảng 4.4, log trace	—
Bộ test ≥12 ca, ảnh, mô phỏng HTML	Hoàn thành	Bảng 4.2–4.3; A01–A09	A10–A11 do SV bổ sung

TÀI LIỆU THAM KHẢO

[1] Giáo trình Cấu trúc dữ liệu và Giải thuật INT1306, tệp Giao-trinh-Cau-truc-du-lieu-va-Giai-thuat-INT1306.pdf; Chương 2, Mục 2.6.2; trang in 50–52; trang PDF 62–64.

[2] Mã mẫu BFS của giảng viên (Ma_BFS_mau.zip) — chỉ tham khảo cách tổ chức module, demo, test và chụp ảnh; thuật toán và kiểm thử là tự cài cho đề DT028.

Mỗi nguồn phải được dùng và dẫn trong nội dung; nguồn mã tham khảo ghi rõ phần tiếp nhận và phần tự làm. Không giữ nguồn không sử dụng.

PHỤ LỤC

PHỤ LỤC A. Tổ chức mã và hướng dẫn chạy

Cây tệp kho mã:

src/DeTai.jl — các hàm tìm biên, tần suất, validator, đối chứng

demo.jl — 8 ca demo + so sánh + nhập bàn phím

test/runtests.jl — 12 ca + 18.594 bất biến + kiểm tra input

scripts/test_catalog.jl, scripts/trace_and_scales.jl

docs/MoPhong_TimBien_DT028.html — trang mô phỏng tương tác

Run_all.ps1 — chạy toàn bộ một lệnh

docs/anh_ket_qua, docs/logs — ảnh và log bằng chứng

Yêu cầu: Julia 1.13.1. Lệnh chạy: julia --startup-file=no demo.jl và test/runtests.jl.

PHỤ LỤC B. Danh mục bằng chứng

Danh mục A01–A09 gồm mã ca, tệp ảnh và log tương ứng; A10–A11 trình bày tại Phụ lục E (Hình 4.10, 4.11). Ảnh trong thân báo cáo phải đọc được.

Mã ảnh	Ca hoặc đối tượng	Tệp ảnh log và SHA
A01	VD01	01_vi_du_1.png
A02	VD02	02_vi_du_2.png
A03	TK01	03_tu_thiet_ke_1.png
A04	TK02	04_tu_thiet_ke_2.png
A05	BIEN01	05_bien_1.png
A06	BIEN02	06_bien_2.png
A07	BIEN03	07_bien_3.png
A08	LOI01	08_input_khong_hop_le.png
A09	TEST_ALL	09_tong_ket_kiem_thu.png
A10	CLONE	Chưa có — sinh viên bổ sung sau khi đẩy GitHub
A11	GITHUB	Chưa có — sinh viên bổ sung sau khi đẩy GitHub

PHỤ LỤC C. Tự kiểm trước khi nộp

Đối chiếu checklist 8 của hướng dẫn; xác nhận thông tin cá nhân/đề, >=12 ca, 11 ảnh, ba quy mô hoặc cấu hình hợp lệ, URL/SHA/permalink, clone sạch và DOCX/PDF cùng phiên bản. Đây là xác nhận sản phẩm đã chuẩn bị, không phải điểm chấm.

□ MSSV và đề tài đúng bảng phân công (N24DCDT034 / DT028).

□ Phần tự cài, tham khảo và công cụ hỗ trợ đã khai báo.

□ ≥12 ca độc lập; đủ ví dụ và ca phiếu; ảnh A01–A09 là ảnh chạy thật trong VS Code.

□ Có truy vết, tính đúng, phân tích thời gian/bộ nhớ và bốn quy mô khảo sát.

Đã có A10/A11, URL, SHA C và permalink; SHA S ghi trong ThongTinNop.txt.

□ Đã thay toàn bộ hướng dẫn/ngoặc vuông; cập nhật trường Word và xuất PDF.

PHỤ LỤC D HƯỚNG DẪN NỘP CLASSROOM

Tiền tố bài cá nhân: N24DCDT034_DT028_PhanDucHuy.

Bảng PL.1 Bốn tệp cần nộp

Tệp	Tên theo mẫu	Nội dung cần có
Word	N24DCDT034_DT028_PhanDucHuy_BaoCao.docx	Bản đã điền; cập nhật mục lục; rubric để GV chấm.
PDF	N24DCDT034_DT028_PhanDucHuy_BaoCao.pdf	Xuất từ chính bản Word nộp.
Mã nguồn	N24DCDT034_DT028_PhanDucHuy_MaNguon.zip	Source, demo, test, README, ảnh/log, mô phỏng HTML.
Thông tin	N24DCDT034_DT028_PhanDucHuy_ThongTinNop.txt	URL GitHub, SHA C/S, permalink, lệnh.

Không nộp nguyên gói mẫu của giảng viên như ZIP cá nhân. Giải nén ZIP của mình ở thư mục mới phải chạy được theo README. Không gói .git, bộ cài, môi trường ảo hoặc cache vào ZIP bài nộp.

C là commit mã/test/input đã chạy và chụp; S là commit chốt bài sau khi thêm ảnh/báo cáo. Clone S sang thư mục mới rồi chạy lại; nếu mã đổi sau C thì cập nhật test/ảnh chịu ảnh hưởng. SHA S ghi trong TXT nộp rời, không phải commit SHA của chính S vào S.

Các thông tin để chép vào TXT

URL GitHub thật: https://github.com/Mildsivan/ctdl-baocao

SHA C đầy đủ: 4277eb8768fbb63ed709df060ecd56e0073346c1 SHA S: ghi trong ThongTinNop.txt (nộp rời)

Permalink hàm chính tại C: https://github.com/Mildsivan/ctdl-baocao/blob/4277eb8768fbb63ed709df060ecd56e0073346c1/src/DeTai.jl ; test: https://github.com/Mildsivan/ctdl-baocao/blob/4277eb8768fbb63ed709df060ecd56e0073346c1/test/runtests.jl

Môi trường và phiên bản: Julia 1.13.1, VS Code 1.142.0, Windows 11

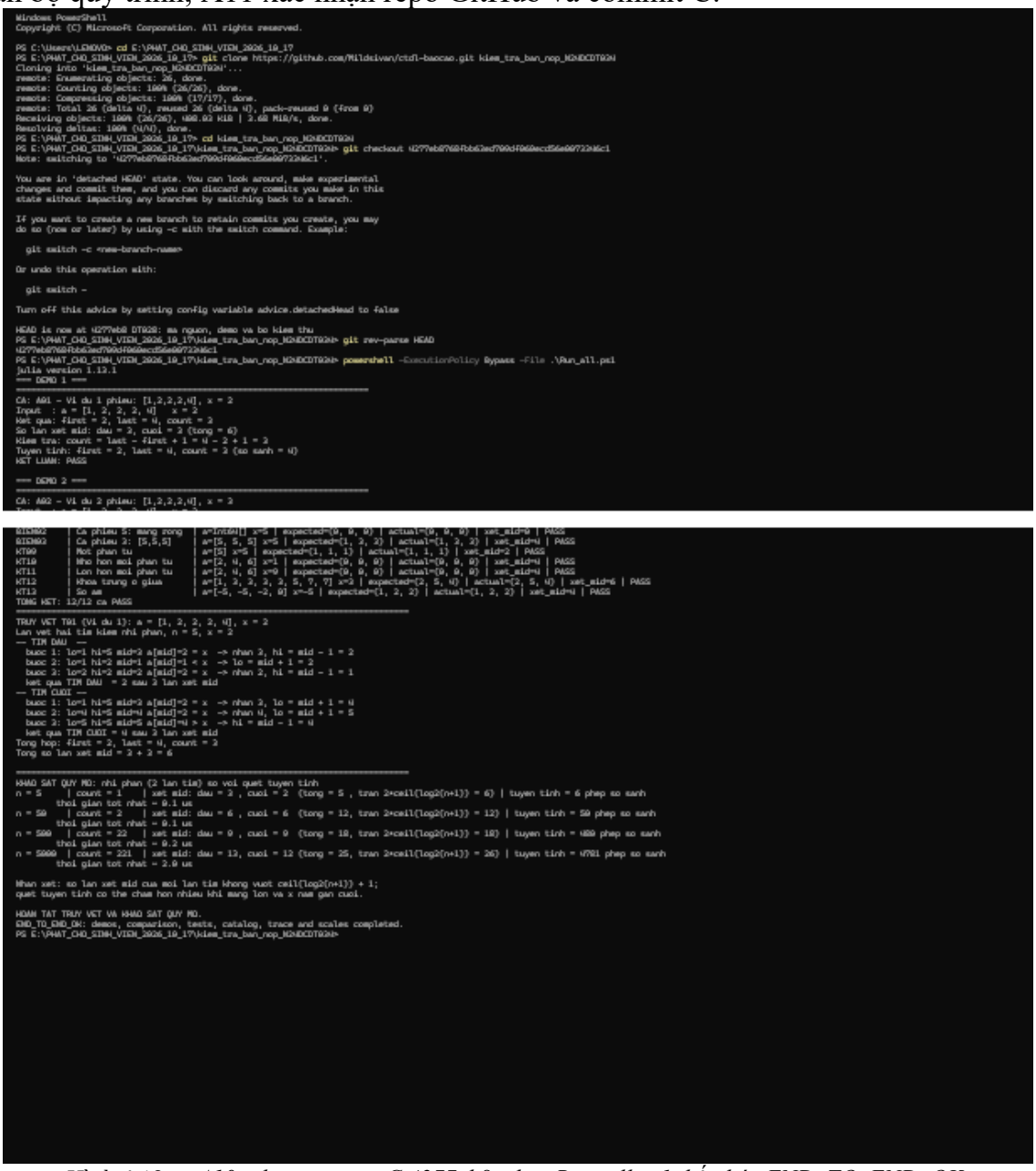
Lệnh chạy hai ví dụ và bộ test: julia --startup-file=no demo.jl --ca=1/--ca=2; julia --startup-file=no test/runtests.jl

Kết quả clone S: ghi trong ThongTinNop.txt (sau khi chốt S) Số ca và phép kiểm: 13 ca / 18.624 phép, 13 PASS

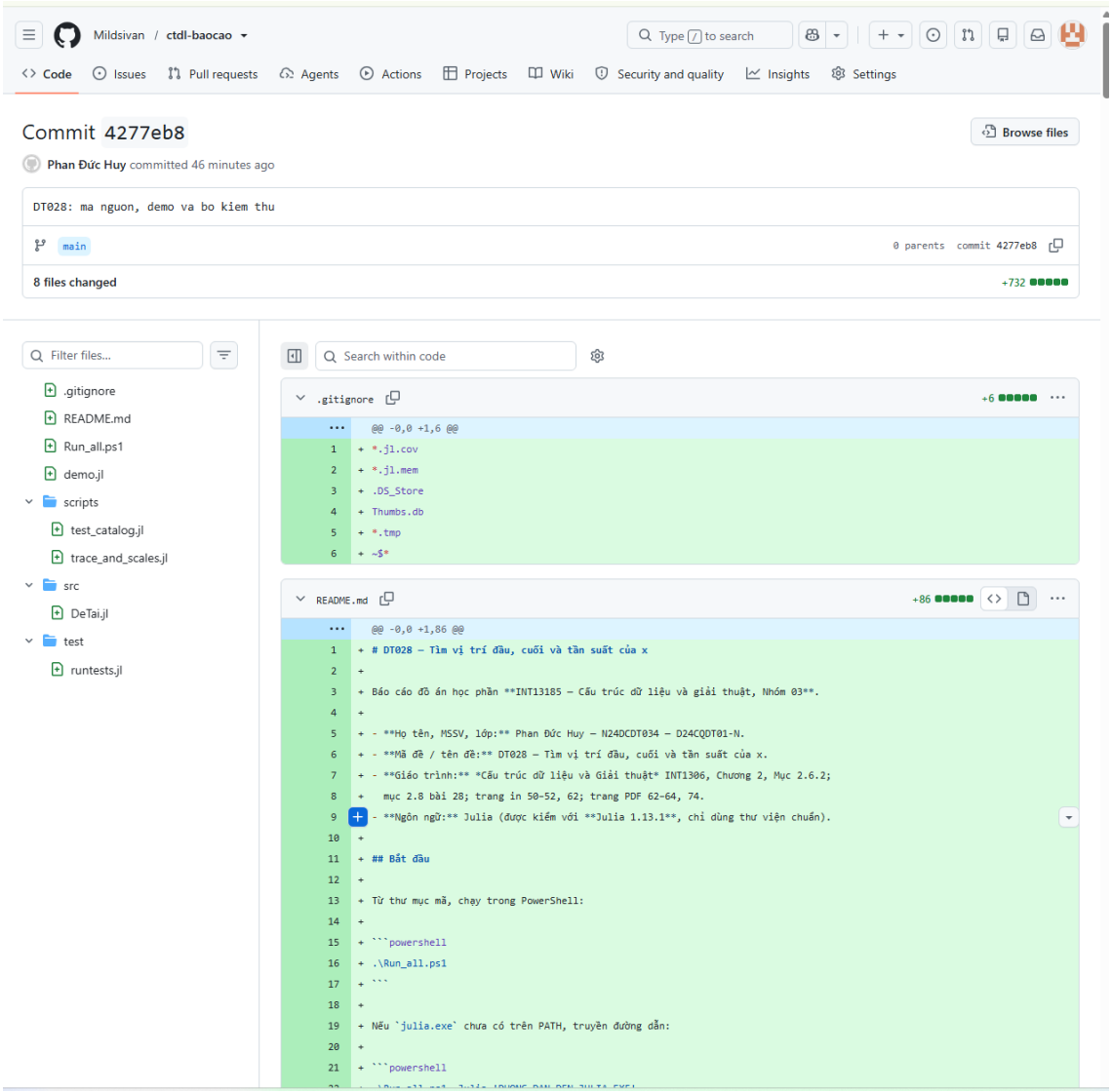
Vào đúng bài tập trên Classroom, đính kèm đủ bốn tệp, mở kiểm tra lại rồi bấm Nộp bài. Kiểm tra trạng thái đã nộp; ảnh rời đã nằm trong báo cáo và ZIP nên không cần gắn lại 11 tệp ảnh riêng.

PHỤ LỤC E — ẢNH MINH CHỨNG A10 VÀ A11

Hai ảnh dưới đây được chụp từ lần chạy thật: A10 kiểm tra clone commit C và chạy lại toàn bộ quy trình; A11 xác nhận repo GitHub và commit C.



Hình 4.10 — A10: clone commit C 4277eb8, chạy Run_all.ps1, kết thúc END_TO_END_OK.



Hình 4.11 — A11: repo GitHub Mildsivan/ctdl-baocao tại commit C 4277eb8.